

Multi-View Evaluation of Microservice Decompositions using Bounded Context and Data Ownership Metrics [Online Appendix]

[Authors anonymized]

No Institute Given

Abstract. This is the online appendix for the paper "Multi-View Evaluation of Microservice Decompositions using Bounded Context and Data Ownership Metrics", containing Semantic Role Labeling background and an extended step-by-step description of the approach using Cargo as a running example, which was excluded from the main manuscript due to space constraints.

A Semantic Role Labeling (SRL) for domain concept extraction

A.1 Background

Semantic Role Labeling (SRL) is a natural-language processing technique that identifies, for each verb (predicate) in a sentence, the participants and their semantic roles. In the PropBank annotation scheme [5] used in our approach, a predicate-argument triple takes the canonical form ($ARG0$, V , $ARG1$), where V is the predicate verb, $ARG0$ is the *proto-agent* (the entity performing or initiating the action), and $ARG1$ is the *proto-patient* (the entity being acted upon). We restrict attention to $ARG0$, V , and $ARG1$ because these roles align most directly with domain *entities* and *responsibilities*.

This alignment is particularly relevant in enterprise requirements engineering, where requirements are commonly written in a subject-action-object style. As a result, the $ARG0/V/ARG1$ structure maps naturally onto the *who does what to what* pattern [2]. $ARG0$ often corresponds to domain *actors*, $ARG1$ to domain *artifacts*, and predicates to the *operations* linking them.

A.2 Running example: JPetStore

Consider the requirement sentence: “*The customer places an order for items from the catalog.*” The BERT-based SRL model [5] identifies:

$ARG0$	V	$ARG1$
<i>the customer places an order</i>		

The argument spans *customer* and *order* become candidate concept mentions, each embedded in its full sentence context. The predicate *places* is recorded as a directed edge label between the two concept nodes weighted by extraction confidence. In the domain graph G_D , this triple produces **customer#0** $\xrightarrow{\text{places}}$ **order#0**, grounding the **Order** bounded context with evidence traceable to source requirements.

However, raw argument spans tend to be noisy: the same entity may appear as *customer*, *customers*, or *the customer account*. Before constructing graph nodes, each span passes through: (1) lowercasing; (2) stopword and determiner removal; (3) lemmatization via spaCy `en_core_web_sm` model; (4) head-noun extraction for multi-word spans; (5) sense mention contextualization: each sense mention is contextualized as "`<NP> / <sentence>`" and encoded with all-MiniLM-L6-v2 (sentence-transformers)¹; (6) sense disambiguation: disambiguation clusters per-lemma embeddings using DBSCAN over cosine distance with the best **eps** selected by silhouette score across four percentile candidates. Small groups and lemmas with too many small clusters receive a single sense (**#0**). Otherwise each cluster receives a sense index *k*, producing identifiers of the form **lemma#k**. In JPetStore, *item* yields **item#0** (Catalog BC) and **item#1** (Cart BC) because the two senses co-occur with structurally distinct requirement vocabularies.

However, SRL alone does not recover all semantically relevant domain relations. In particular, SRL models tend to underrepresent relations expressed through nominalizations [4], implicit dependencies [3], coordination structures, and attribute-style descriptions that do not instantiate explicit predicate frames. Consequently, relying exclusively on (*ARG0*, *V*, *ARG1*) triples may yield sparse or incomplete domain graphs.

To complement these limitations, a second extraction phase uses an LLM (LLaMA-3.3-70B²) to normalize and augment relations from each requirement. The LLM is prompted with the requirement text together with its SRL frames, enabling it to infer additional semantic relations beyond those captured by explicit verbal predicates. It returns typed relations (subject, predicate, object, type $\in \{\textit{action}, \textit{composition}, \textit{attribute}, \textit{dependency}\}$) with confidence scores. LLM-produced entities are aligned to existing senses by cosine similarity (threshold 0.75); entities that do not match any known sense above a relaxed fallback threshold are added as new nodes tagged **#new_<uuid>**.

A.3 Limitations of relation extraction

The extraction pipeline combines SRL-based predicate analysis with LLM-based relation augmentation. Nevertheless, several limitations remain that affect the quality of the resulting domain graph G_D . SRL models may fail to instantiate separate predicate frames for coordinated predicates (e.g., *validates and processes the payment*"), causing relations associated with secondary predicates to become

¹ <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

² https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_3/

underrepresented. Passive constructions with implicit agents can produce isolated *ARG1* nodes, while nominalized expressions (e.g., *order placement*) are frequently omitted by verbal SRL models because no explicit predicate frame is generated. In addition, domain-specific jargon absent from the SRL pretraining corpus may yield low-confidence span boundaries, leading to truncated or incorrectly merged concepts after canonicalization.

The LLM augmentation phase partially mitigates these issues by inferring implicit relations, recovering nominalized actions, and normalizing semantically equivalent expressions across heterogeneous requirement styles. However, this introduces a complementary set of risks. Although the LLM augmentation phase uses structured output constraints (a typed Pydantic schema with confidence scores and evidence spans), the model may still hallucinate unsupported relations or overgeneralize weak associations, since schema compliance does not guarantee factual grounding in the source requirements.

To reduce the impact of sparse or noisy relations, seed-anchored label propagation (step B.2 of the proposed approach) re-anchors seeded nodes to their known bounded context (BC) assignments. The sensitivity of downstream metrics to extraction quality is further moderated by the teleport mixing rate and seed anchoring strength, discussed in Section B.4. Nevertheless, projects characterized by extensive nominalization, implicit-agent passives, or highly specialized terminology remain at elevated risk of noisy domain graphs, and manual inspection of the extracted G_D is therefore recommended.

B Approach

Cargo³ is a reference implementation of Domain-Driven Design patterns [1]. Its four bounded contexts are **Booking** (cargo lifecycle and itinerary assignment), **Handling** (physical cargo events), **Routing** (pathfinding and voyage management), and **Tracking** (delivery status monitoring). In the following subsections, we describe each step of our approach using Cargo as a running example.

B.1 Multi-view architectural modeling

The approach begins by constructing three independent architectural views from the monolithic system’s source artifacts. We describe each in turn and note the practical choices made during extraction.

Domain view. SRL over the Cargo requirements corpus [6] produces a directed domain graph G_D with 27 concept senses. High-confidence senses include **event#0**, **voyage#0**, **cargo#1**, **itinerary#1**, **route specification#0**, and **carrier movement#0**. After performing sense disambiguation, two distinct terms surface per sense. **carrier#0** and **carrier#1** distinguish schedule-management usage (co-occurring with departure/arrival timetables) from delivery-confirmation usage (co-occurring

³ <https://github.com/citerus/dddsample-core>

with cargo movement verbs). `location#0` and `location#1` distinguish geographic identifiers from the operational notion of a cargo’s current transit location. `delivery#0` and `delivery#1` separate the physical delivery act (handling event) from the delivery status.

Edge weights in G_D are set to SRL (product of predicate and argument span probabilities) and LLM extraction confidence, encoding uncertainty directly in the graph structure used for label propagation.

Implementation view. The implementation graph G_I contains 91 class nodes spanning six packages: `domain.model`, `application`, `interfaces`, `infrastructure`, and the `com.pathfinder` sub-system. Each class node is enriched with a semantic descriptor generated by Llama 3.1 8B, prompting the model with the class name, package path, field declarations, and public method signatures. The prompt instructs the model to produce a two-to-three sentence summary of the class’s domain responsibility in plain language; the resulting text is encoded with a sentence encoder to produce the embedding used in s^{emb} .

Two LLM-specific threats affect descriptor quality. First, hallucination affects classes with minimal implementations or generic names: `SampleDataGenerator` and `SampleVoyages` receive descriptors referencing voyage and cargo concepts drawn from name fragments rather than actual behavior, so their `INFRAFILTER` exclusion is essential. Second, context-window truncation affects large classes (e.g., `BookingServiceFacadeImpl` with many delegating methods): we mitigate this by ordering the prompt to place public interface methods and constructors first and capping descriptor length at 100 tokens.

Centrality vectors (degree, betweenness, closeness) are computed on the class-level dependency projection and percentile-normalized, mean-centered before structural comparison. This normalization makes the structural signal invariant to graph scale, enabling comparison across systems of different sizes.

Data view. Cargo’s schema contains six tables: `CARGO`, `CARRIER_MOVEMENT`, `HANDLING_EVENT`, `LEG`, `LOCATION`, and `VOYAGE`. Static analysis of DAO method calls and JPA `@Entity` / `@Column` mappings links eleven classes to tables. Tables are converted to nodes into the data graph D_T . Each table node also receives an LLM-generated descriptor for use in the I→T alignment direction; these descriptors are generated from table name, column list, and inferred FK structure.

B.2 Inducing BCs from the domain view

Starting from expert-annotated seed sets for each BC, semi-supervised label propagation on G_D assigns every concept node v a probability distribution $P_v : \mathcal{B} \rightarrow [0, 1]$ over the four BCs. At each iteration, each non-seeded node aggregates its neighbours’ distributions weighted by edge confidence and mixes the result with a uniform prior:

$$\tilde{P}_v(b) = (1 - \gamma) \frac{\sum_{u \in \mathcal{N}(v)} w_{vu} P_u(b)}{\sum_{u \in \mathcal{N}(v)} w_{vu}} + \gamma \pi(b).$$

Seeded nodes are partially re-anchored to their seed distribution via λ , preserving expert knowledge throughout propagation.

For Cargo the seed sets are: $\text{booking} \leftarrow \{\text{cargo}\#1, \text{itinerary}\#1, \text{route specification}\#0, \text{delivery}\#0\}$; $\text{handling} \leftarrow \{\text{event}\#0, \text{location}\#0, \text{location}\#1\}$; $\text{routing} \leftarrow \{\text{voyage}\#0, \text{route}\#1, \text{carrier movement}\#0, \text{destination}\#1\}$; $\text{tracking} \leftarrow \{\text{status}\#0, \text{estimate delivery date}\#0\}$.

Table 1 reports the resulting $P(\text{BC} \mid c)$ for the most informative concept nodes.

Table 1. Label-propagated BC distributions for selected Cargo concept nodes. Bold value indicates the dominant BC (> 0.5). BCs: Bkg = booking, Hnd = handling, Rte = routing, Trk = tracking.

Concept	Bkg	Hnd	Rte	Trk	Dominant
event#0	.019	.940	.020	.021	handling
cargo#1	.907	.020	.053	.020	booking
route#1	.021	.023	.893	.062	routing
voyage#0	.050	.019	.911	.019	routing
itinerary#1	.943	.019	.019	.019	booking
route specification#0	.943	.019	.019	.019	booking
estimate delivery date#0	.020	.021	.043	.916	tracking
status#0	.021	.022	.037	.920	tracking
carrier movement#0	.021	.021	.938	.021	routing
carrier#1	.072	.086	.243	.598	tracking
delivery#1	.062	.067	.349	.521	tracking
location#0	.021	.930	.028	.021	handling

B.3 Cross-view probabilistic alignment

The three views provide complementary evidence at different abstraction levels but are not directly comparable. For each ordered pair of nodes c and u from different graphs (G_D , G_I , G_T), a composite alignment score combines three signals:

$$s(c, u) = \alpha_{emb} s^{emb}(c, u) + \alpha_{struct} s^{struct}(c, u) + \alpha_{data} s^{data}(c, u), \quad \sum_l \alpha_l = 1.$$

The *semantic signal* s^{emb} is the cosine similarity between the sentence-encoded descriptors of c and u ; it aligns nodes sharing naming and responsibility conventions. The *structural signal* s^{struct} is the cosine similarity between percentile-normalised, mean-centred centrality vectors (degree, betweenness, closeness) of c in G_D and u in G_I ; it can be negative when their topological roles are opposed (a hub concept matched with a peripheral class, or vice versa). The *data signal* s^{data} measures table-access overlap between the tables accessed by u and those associated with c through its aligned implementation nodes; it is non-zero only

for classes with direct persistence access and applies only to the $I \rightarrow D$ and $I \rightarrow T$ alignment directions.

Each class u retains its top- $k = 10$ concept candidates; a softmax over composite scores produces the probabilistic alignment $P(c \mid u)$. We illustrate the mechanism through four contrasting examples.

*Consistent alignment: **HandlingEvent**.* This class records physical cargo events (loading, unloading, customs, delivery). All three signals converge on **event#0**: $s^{emb} = 0.534$ (highest semantic similarity for this class), $s^{struct} = 0.642$ (both **event#0** and **HandlingEvent** are high-betweenness hubs in their respective graphs), and $s^{data} = 0.5$ (the class maps directly to **HANDLING_EVENT**). The resulting distribution assigns $P(\text{event\#0} \mid \text{HandlingEvent}) = 0.737$ with runner-up **date#0** at 0.100, driven by date-related attributes on the event entity. This is the ideal case: all three signals agree and provide independent confirmation.

*Semantic-structural discrepancy: **Voyage**.* The **Voyage** class encapsulates a carrier’s schedule. Semantically, $s^{emb}(\text{voyage\#0}, \text{Voyage}) = 0.635$ is the top score, as expected from the name match. However, $s^{struct}(\text{voyage\#0}, \text{Voyage}) = -0.870$: **voyage#0** has high degree centrality in G_D (extensively cross-referenced in requirements), while **Voyage** in G_I is a low-betweenness leaf node referenced only by **CarrierMovement** and repository interfaces. The structural signal strongly disfavours the semantic alignment. By contrast, $s^{struct}(\text{carrier\#1}, \text{Voyage}) = 0.528$: both occupy peripheral positions in their respective graphs. The data signal partially counteracts the structural penalty ($s^{data}(\text{voyage\#0}, \text{Voyage}) = 0.5$ via the **VOYAGE** table), but is insufficient to overcome the structural mismatch. The final distribution assigns $P(\text{carrier\#1} \mid \text{Voyage}) = 0.300$ and $P(\text{voyage\#0} \mid \text{Voyage}) = 0.175$. This is architecturally interpretable: requirements describe voyages through the lens of carrier service scheduling, and the structural signal correctly captures this usage even when the name suggests a different alignment.

*Structural dominance: **Delivery**.* This value object aggregates transport status, ETA, and routing compliance for a **Cargo**. It presents the clearest case of the structural signal reversing the semantic ranking. Semantically, $s^{emb}(\text{delivery\#1}, \text{Delivery}) = 0.668$. But $s^{struct}(\text{delivery\#1}, \text{Delivery}) = -0.463$: the **delivery#1** concept is peripheral in G_D (low betweenness), while **Delivery** is a high-betweenness hub in G_I , referenced by **Cargo**, **HandlingEventFactory**, multiple application services, and booking DTOs. The centrality mismatch overrides the semantic agreement. In contrast, $s^{struct}(\text{carrier movement\#0}, \text{Delivery}) = 0.593$: carrier movements are the primary drivers of delivery status updates, and both occupy similar hub positions. Since **Delivery** has no direct table mapping ($s^{data} = 0$ everywhere), the final distribution is highly dispersed: $P(\text{carrier movement\#0} \mid \text{Delivery}) = 0.196$, $P(\text{carrier\#0} \mid \text{Delivery}) = 0.149$, $P(\text{delivery\#0} \mid \text{Delivery}) = 0.142$, $P(\text{delivery\#1} \mid \text{Delivery}) = 0.114$, with no concept reaching the Ψ assignment threshold (> 0.30). The near-uniform distribution correctly flags **Delivery** as a genuinely cross-cutting class whose BC entropy will be high regardless of where it is placed.

Anti-corruption layer: ExternalRoutingService. This class translates `RouteSpecification` objects into pathfinder graph queries. The top semantic candidates are $s^{emb}(\text{route specification}\#0) = 0.466$ and $s^{emb}(\text{route}\#1) = 0.389$; the structural signal is positive for both (both concept and class occupy bridge positions in their respective graphs); and $s^{data} = 0$ (no direct table access — the class delegates persistence to repository interfaces). The final distribution $P(\text{route}\#1) = 0.486$, $P(\text{route specification}\#0) = 0.311$ correctly identifies a class that both satisfies booking-BC specifications and returns routing-BC candidates. The absence of a data signal is not a failure; it signals that this class is a pure coordination layer with no persistence obligation.

Parameter guidance. The signal weights $\alpha_{emb} = 0.5$, $\alpha_{struct} = 0.3$, $\alpha_{data} = 0.2$ are a reasonable default. The semantic signal should carry the largest weight because LLM descriptors are informative for the majority of domain classes; reducing α_{emb} below 0.4 risks over-weighting topological coincidences. The structural signal weight should be increased (toward 0.4) for codebases with weak naming conventions or generated code, where the semantic signal is noisy. The data signal weight should remain modest (≤ 0.25) because it is zero for more than half of all classes; a large α_{data} creates inconsistent alignment quality across architectural layers. For the top- k filtering: $k = 10$ candidates per class before softmax removes long-tail noise without truncating genuine secondary alignments; for service-level BC computation (Section B.4) we use $k = 1$ (dominant concept only) to produce sharper per-service BC distributions.

B.4 Architectural metrics

Bounded context alignment (BCA) metric The alignment distributions $P(c | u)$ propagate BC knowledge from concept nodes to implementation classes via:

$$P(\text{BC} | u) = \frac{1}{Z_u} \sum_{c \in \text{top-}k(u)} P(c | u) P(\text{BC} | c).$$

Table 2 reports the resulting distributions (computed with $k=3$) for twelve representative classes alongside their normalised entropy H .

Several findings from Table 2 cannot be recovered from structural metrics. `VoyageRepository` concentrates in the Routing BC ($P = 0.909$): the pipeline correctly infers that voyages are routing infrastructure, not booking objects, consistent with the label propagation result for `voyage\#0`. The four lowest-entropy classes (`HandlingEvent`, `HandlingEventServiceImpl`, `VoyageRepository`, `CargoRepository`, all $H < 0.36$) are reliable single-BC anchors for decomposition evaluation. `Itinerary` ($H = 0.60$) and `Delivery` ($H = 0.67$) are both routing-dominant because the itinerary is produced by the routing pathfinder and the delivery status aggregates carrier-movement schedules; neither can be co-assigned with purely Booking BC classes without increasing service entropy. Finally, `CarrierMovement` reaches near-maximum entropy ($H = 0.92$): `Routing` (0.354), `Tracking` (0.296), and `Booking` (0.280) are nearly equally weighted, meaning this class cannot be cleanly assigned to any BC in any decomposition.

Table 2. $P(\text{BC} \mid u)$ for representative Cargo classes (top- $k=3$ concept aggregation). H : normalised entropy ($H=0$: single BC; $H=1$: uniform). Dominant concept weight in parentheses.

Class	Dominant concept	Hnd	Bkg	Rte	Trk	H
HandlingEvent	event#0 (0.737)	.884	.019	.021	.075	0.33
HndlEventSvcImpl	event#0 (0.853)	.876	.078	.024	.022	0.35
VoyageRepository	voyage#0 (0.644)	.019	.053	.909	.019	0.28
CargoRepository	cargo#1 (0.910)	.021	.881	.077	.021	0.34
RouteSpecification	route#1 (0.467)	.023	.121	.674	.182	0.66
ExternalRoutingSvc	route#1 (0.486)	.022	.416	.519	.044	0.67
Itinerary	route#1 (0.236)	.021	.269	.670	.039	0.60
Delivery	carrier mvmt#0 (0.196)	.040	.309	.611	.040	0.67
TrackingId	status#0 (0.482)	.329	.104	.047	.521	0.78
CargoInspectionSvc	system#0 (0.361)	.047	.759	.099	.096	0.58
BookingServiceImpl	delivery#1 (0.385)	.157	.066	.476	.301	0.85
CarrierMovement	carrier#1 (0.268)	.070	.280	.354	.296	0.92

In particular, for a decomposition m consisting of k services, the service-level BC distribution is the mean of its member class distributions and $H(S_i)$ is its normalised entropy. The decomposition-level penalty $\text{Pen}_{bc}(m) = \frac{1}{|V_C|} \sum_i |U_i| H(S_i)$ is the size-weighted mean, and NMI is computed between service membership and dominant-BC assignment. Evaluating 247 Cargo candidate decompositions ($k \in [6, 33]$), NMI ranges from 0.098 to 0.330 and Pen_{bc} from 0.641 to 0.894. Table 3 contrasts the extremes.

Table 3. Top-ranked and bottom-ranked Cargo decompositions under BCA (247 candidates). $\text{score_rel} = 1 - \text{Pen}_{bc}/H_{\text{global}}$; *pure*: services with $H < 0.50$.

Decomposition ID	k	NMI	Pen_{bc}	score_rel	<i>pure</i>
04_03_09_48_11_824424 (rank 1)	33	0.330	0.641	0.343	17/33
04_03_10_03_04_859707 (rank 2)	19	0.312	0.684	0.299	7/19
04_03_10_05_59_684894 (rank 3)	19	0.293	0.701	0.282	6/19
04_03_09_48_12_076197 (rank 245)	6	0.124	0.877	0.101	0/6
04_03_09_55_48_471243 (rank 247)	6	0.098	0.894	0.085	0/6

The top decomposition ($k=33$, $\text{NMI} = 0.330$) achieves 17/33 pure services: fine granularity allows high-entropy classes (**Delivery**, **BookingServiceImpl**, **CarrierMovement**) to be isolated into dedicated micro-services where their mixing does not inflate the entropy of otherwise-pure clusters. The handling cluster achieves $H \approx 0.35$; the routing cluster (**VoyageRepository**, **Voyage**, **CarrierMovement**) achieves $H \approx 0.65$, bounded below by **CarrierMovement**'s irreducible individual

entropy of 0.92. The bottom decomposition ($k=6$, $NMI = 0.098$) produces no pure services.

Data ownership (DO) metric The data ownership metric operates entirely on statically extracted persistence artifacts and is independent of the alignment parameters. It consists of two parts: identifying transactional scopes and computing pairwise penalties based on shared table access and transactional co-participation.

Transactional scope extraction. **@Transactional**-annotated root methods are identified by AST inspection; breadth-first traversal of the call graph from each root (depth $d \leq 2$) yields the set of classes participating in that transaction. Table 4 decodes the six transactions found in Cargo from their bitmask representation.

Table 4. Transactional scopes in Cargo (`class_tx_bits`, 6 roots, one bit per root). Abbreviations — *BkS*: `BookingServiceImpl`; *CR*: `CargoRepository`; *C*: `Cargo`; *LR*: `LocationRepository`; *CF*: `CargoFactory`; *RS*: `RouteSpecification`; *It*: `Itinerary`; *D*: `Delivery`; *Rt*: `RoutingService`; *HES*: `HandlingEventServiceImpl`; *HEF*: `HandlingEventFactory`; *AE*: `ApplicationEvents`; *HER*: `HandlingEventRepository`; *HH*: `HandlingHistory`; *CI*: `CargoInspectionServiceImpl`.

Tx	Inferred root operation	Participating classes	Bit
0	<code>bookNewCargo</code>	BkS, C, CR, CF, LR	0b000001
1	<code>requestPossibleRoutes</code>	BkS, C, CR, Rt	0b000010
2	<code>assignCargoToRoute</code>	BkS, C, CR, It, D	0b000100
3	<code>changeDestination</code>	BkS, C, CR, RS, D, LR	0b001000
4	<code>registerHandlingEvent</code>	HES, HEF, AE, HER	0b010000
5	<code>inspectCargo</code>	CI, C, CR, AE, HER, HH, D	0b100000

Two structural properties are immediately visible. First, Tx0–3 and Tx5 share a booking core: `Cargo` and `CargoRepository` appear in five of the six transactions. Any split of these classes across services creates services that co-participate in the same transactional scopes, requiring distributed coordination for five out of six user-initiated operations. Second, Tx4 is fully disjoint: it involves only Handling-BC classes (`HandlingEventServiceImpl`, `HandlingEventFactory`, `ApplicationEvents`, `HandlingEventRepository`) and shares no participants with Tx0–3. A decomposition that co-locates all handling-event classes in a single service will therefore always achieve $T(S_{\text{hnd}}) \cap T(S_j) = \emptyset$ for any booking-aligned service S_j , yielding zero DO penalty for that pairing. This structural isolation of Tx4 is why the vast majority of Cargo decompositions are penalty-free.

Table access and pairwise penalties. The pairwise DO penalty is:

$$\text{pen}_{\text{data}}(S_i, S_j) = J(\text{Access}(S_i), \text{Access}(S_j)) \cdot J(T(S_i), T(S_j)),$$

where J is the Jaccard similarity. The multiplicative form requires simultaneous table-access overlap and transactional co-participation to produce a non-zero signal: shared read access to `LOCATION` (a lookup table read by many services) produces $J_{\text{acc}} > 0$ but $J_{\text{tx}} = 0$, hence zero penalty; `ApplicationEvents` co-participating in Tx4 and Tx5 creates transactional overlap but between services with disjoint table access, again yielding zero penalty. Only genuine write-ownership conflicts (where two services both write to the same table *and* are invoked within the same transactional scope) trigger a non-zero signal. Of the 247 Cargo decompositions, 244 incur $\text{Pen}_{\text{data}} = 0$. The three violations are reported in Table 5.

Table 5. Non-zero data ownership penalties in Cargo. J_{acc} : table-access Jaccard; J_{tx} : transactional-scope Jaccard; $\text{pen}_{\text{data}} = J_{\text{acc}} \times J_{\text{tx}}$.

Decomposition	k	NMI	Violating pair	J_{acc}	J_{tx}
04_03_09_39_14_406060	14	0.208	(S_0, S_5)	0.50	0.33
04_03_09_43_04_295983	14	0.232	(S_0, S_7)	0.50	0.83
04_03_10_03_04_783797	20	0.253	(S_1, S_7)	0.50	0.83

All three violations share $J_{\text{acc}} = 0.50$: one service accesses $\{\text{CARGO}, X\}$ while the other accesses only $\{\text{CARGO}\}$, so the intersection is $\{\text{CARGO}\}$ and the union has size 2. This pattern arises when a decomposition assigns `CargoRepository` (which also transitively resolves `LOCATION` or `LEG` via query methods) to a different service from auxiliary booking classes such as `CargoInspectionServiceImpl` or `CargoFactory`, both of which also access `CARGO` through ORM mappings. In the severe cases ($J_{\text{tx}} = 5/6 \approx 0.83$), the two services co-participate in five of the six transactions (every booking operation and the cargo inspection transaction) meaning that virtually every user-initiated action would require cross-service distributed coordination. The milder case ($J_{\text{tx}} = 1/3$) shares table ownership but limits transactional co-occurrence to two operations, confining the coordination burden.

Parameter summary and sensitivity

Table 6 consolidates all tuneable parameters with recommended ranges and the metric most affected by each.

Label propagation sensitivity. BC seeds were derived from published reference architectures for each subject system. For systems without documented architectures, seed definition requires domain expertise and introduces subjectivity; the sensitivity of downstream NMI rankings to seed coverage was therefore evaluated explicitly.

Three seed-set variants were constructed per system: *full* (all annotated seeds), *restricted* (at most 2 seeds per BC, selecting the first available node

Table 6. Pipeline parameters, recommended ranges, and primary sensitivity. Sensitivity assessed qualitatively on Cargo decomposition rankings.

Parameter	Role	Recommended range
γ	LP uniform prior weight	[0.05, 0.15]
λ	LP seed re-anchoring	[0.30, 0.50]
α_{emb}	Semantic signal weight	[0.40, 0.60]
α_{struct}	Structural signal weight	[0.20, 0.40]
α_{data}	Data signal weight	[0.10, 0.25]
k (softmax)	Top- k candidates	10
k (BC agg.)	Concepts for $P(\text{BC} \mid u)$	3
τ	Alignment threshold	0.25–0.40

per BC from the full set), and *extended* (full seeds augmented with manually identified boundary-concept nodes). For each variant, label propagation was re-run with identical hyperparameters ($\gamma = 0.12$, $\lambda = 0.92$, 60 iterations), and NMI was computed over the same candidate decomposition pool. Spearman ρ was used to compare NMI rankings across variants.

Results are summarised in Table 7. For Cargo, rankings were stable across all variant pairs ($\rho \geq 0.85$, $p < 0.001$), with 4–5 of the top-10 decompositions shared between full and restricted sets. For DayTrader, full and extended seeds produced identical results ($\rho = 1.00$); reducing to 2 seeds per BC degraded mean NMI from 0.33 to 0.20 and weakened ranking correlation ($\rho = 0.66$). JPestore exhibited the highest sensitivity ($\rho = 0.16$, $p = 0.065$), with no overlap in the top-10 ranked decompositions between full and restricted variants. JPestore is also the system with the smallest domain graph (27 concept nodes, 4 BCs), suggesting that restricted seeds are insufficient when the ratio of seeded nodes to total concept nodes is low.

These results indicate a minimum coverage threshold: with fewer than approximately 3 seeds per BC, propagation may fail to overcome the uniform teleport prior and produce unstable BC assignments. For systems with well-documented reference architectures, the full seed set is recommended; for novel systems, we recommend at least 3 seeds per BC and post-hoc inspection of the *ambiguous_nodes* diagnostic (nodes where the top-BC probability < 0.60 or the gap to the second-ranked BC < 0.15).

Label propagation parameters (γ , λ) were set empirically. $\gamma \in [0.05, 0.15]$ controls the uniform prior mixing rate: values below 0.05 risk degenerate one-BC collapse in sparse graphs, while values above 0.20 wash out BC boundaries. $\lambda = 0.92$ was found to preserve seed annotations reliably across all three systems; lower values ($\lambda \in [0.3, 0.5]$) may be appropriate when seed nodes are known to straddle BC boundaries.

Table 7. Sensitivity of NMI rankings to seed-set variant. ρ : Spearman correlation between per-decomposition NMI vectors. Mean NMI reported over all candidate decompositions per system.

System	Variant	Seeds	Mean NMI	ρ vs full	p
Cargo	full	19	0.218	—	—
	restricted	8	0.208	0.905	<0.001
	extended	21	0.246	0.928	<0.001
DayTrader	full	17	0.329	—	—
	restricted	8	0.200	0.664	<0.001
	extended	24	0.329	1.000	<0.001
JPetstore	full	23	0.597	—	—
	restricted	8	0.646	0.157	0.065
	extended	23	0.597	1.000	<0.001

C Data Ownership metric: DayTrader example

This section provides the detailed data ownership analysis summarised in the main paper’s RQ3. DayTrader is a Java EE stock-trading benchmark application with 26 classes, 6 database tables (`ACCOUNT`, `ACCOUNTPROFILE`, `HOLDING`, `KEYGEN`, `ORDER`, `QUOTE`), and 29 `@Transactional` root methods, all residing in `TradeSLSBBean` except for two message-driven bean handlers (`DTBroker3MDB.onMessage`, `DTStreamer3MDB.onMessage`) and one singleton initializer (`MarketSummarySingleton.setup`). The `INFRAFILTER` step excludes two utility classes — `Log` and `TradeConfig` — prior to all metric computations.

C.1 ALPHA vs. BETA: Overview

Expert validation asked ten practitioners to assess data ownership coherence between two decompositions. ALPHA is the best-ranked decomposition by Pen_{data} (0.042, 2 penalized service pairs) and BETA is the worst-ranked (2.582, 12 penalized pairs); both produce 9 and 8 services respectively, so granularity does not explain the difference. Seven of ten evaluators preferred ALPHA, citing reduced transactional overlap and clearer service autonomy. Table 8 summarises the key metrics for both.

C.2 Service compositions

Tables 9 and 10 list the classes in the services relevant to the penalized pairs. Classes excluded by `INFRAFILTER` (`Log`, `TradeConfig`) are omitted.

The critical structural difference is visible immediately: ALPHA places `TradeSLSBBean` and `TradeDirect` in the same service (S_1), while BETA separates them: `TradeSLSBBean` into S_6 and `TradeDirect` into S_4 . These two classes implement the same `TradeServices`

Table 8. ALPHA and BETA decompositions in DayTrader. k : number of services. Pen_{data} : total data ownership penalty. $|\mathcal{P}|$: number of penalized service pairs. NMI computed against dominant-BC assignment after INFRAFILTER.

Decomp.	k	NMI	Pen_{data}	$ \mathcal{P} $
ALPHA	9	0.440	0.042	2
BETA	8	0.392	2.582	12

Table 9. ALPHA service composition (services relevant to penalized pairs).

Service Classes			
S_1	TradeSLSBBean, TradeAction, CompleteOrderThread,	TradeSLSBLocal, TradeServices, RunStatsDataBean	TradeDirect, OrderDataBean,
S_3	MarketSummarySingleton, MarketSummaryDataBean		
S_7	DTBroker3MDB, DTStreamer3MDB, TimerStat, MDBStats		

interface: `TradeSLSBBean` is the EJB session-bean facade that wraps every trading operation as a managed transaction, and `TradeDirect` is the JDBC implementation to which `TradeSLSBBean` delegates. Splitting them across services creates a design where every `@Transactional` call in S_6 immediately crosses into S_4 , making distributed coordination mandatory for all standard trading operations.

C.3 Pen_{data} computation: ALPHA (S_1, S_7)

ALPHA’s two penalized pairs are (S_1, S_7) and (S_1, S_3) , with penalties of 0.036 and 0.006 respectively. We walk through (S_1, S_7) as the larger of the two.

Table access. Both S_1 and S_7 access all six DayTrader tables: $\text{Access}(S_1) = \text{Access}(S_7) = \{\text{ACCOUNTEJB}, \text{ACCOUNTPROFILEEJB}, \text{HOLDINGEJB}, \text{KEYGENEJB}, \text{ORDEREJB}, \text{QUOTEJB}\}$. S_1 accesses all six tables because `TradeDirect` performs JDBC operations across the full schema for buying, selling, and account management. S_7 accesses all six because the MDB order-completion handlers (`DTBroker3MDB`, `DTStreamer3MDB`) process completed trade orders end-to-end. Hence:

$$J_{\text{acc}}(S_1, S_7) = \frac{|\{6\}|}{|\{6\}|} = 1.0.$$

Transactional scopes. $T(S_1)$ is derived from the `@Transactional` roots reachable within S_1 ’s class set, covering the majority of the 29 transaction roots via `TradeSLSBBean` and its collaborators. $T(S_7)$ is derived from the two MDB entry points (`DTBroker3MDB.onMessage`, `DTStreamer3MDB.onMessage`). Although the MDB handlers also touch trading data, they operate asynchronously and their

Table 10. BETA service composition (services involved in the worst penalized pair).

Service Classes	
S_4	<code>TradeDirect</code> , <code>KeySequenceDirect</code> (+ <i>Log</i> , <i>excluded</i>)
S_6	<code>TradeSLSBBean</code> , <code>TradeSLSBRemote</code> , <code>TradeSLSBLocal</code>

transactional scopes overlap minimally with the synchronous EJB transactions in S_1 . The resulting Jaccard is:

$$J_{tx}(S_1, S_7) = \frac{|T(S_1) \cap T(S_7)|}{|T(S_1) \cup T(S_7)|} = 0.036,$$

yielding $\text{pen}_{data}(S_1, S_7) = 1.0 \times 0.036 = 0.036$.

Interpretation. Despite full table-access overlap, the penalty is near zero because the two services participate in almost entirely disjoint transactional scopes: S_1 handles synchronous user requests while S_7 handles asynchronous JMS-triggered completions. The multiplicative form correctly reflects that this shared table access does not translate into a coordination obligation under normal operation.

The second penalized pair, (S_1, S_3) , has $J_{acc}(S_1, S_3) = 1/6 \approx 0.167$ (the market-summary service accesses only `QUOTEJJB`) and $J_{tx}(S_1, S_3) \approx 0.038$, yielding $\text{pen} = 0.006$. ALPHA’s total $\text{Pen}_{data} = 0.036 + 0.006 = 0.042$.

C.4 Pen_{data} computation: BETA (S_4, S_6)

BETA’s worst penalized pair is (S_4, S_6) with penalty 0.793, which alone accounts for 30.7% of its total $\text{Pen}_{data} = 2.582$.

Table access. Both S_4 and S_6 access all six tables: $\text{Access}(S_4) = \text{Access}(S_6) = \{\text{ACCOUNTJJB}, \text{ACCOUNTPROFILEJJB}, \text{HOLDINGJJB}, \text{KEYGENJJB}, \text{ORDERJJB}, \text{QUOTEJJB}\}$. S_6 accesses all six through `TradeSLSBBean`’s 24 `@Transactional` root methods. S_4 accesses all six through `TradeDirect`, which performs the actual JDBC reads and writes for every trading operation. Hence:

$$J_{acc}(S_4, S_6) = 1.0.$$

Transactional scopes. $T(S_6)$ is the set of 24 `@Transactional` root methods on `TradeSLSBBean`: `buy`, `sell`, `cancelOrder`, `getOrders`, `login`, `register`, and 18 others. $T(S_4)$ contains 28 of the 29 transaction roots reachable from S_4 ’s classes within call depth 2. The intersection $T(S_4) \cap T(S_6)$ contains 23 of the 29 total transaction roots:

$$J_{tx}(S_4, S_6) = \frac{23}{29} \approx 0.793.$$

The pairwise penalty is therefore:

$$\text{pen}_{data}(S_4, S_6) = 1.0 \times 0.793 = 0.793.$$

Interpretation. This is the most severe violation in the DayTrader evaluation. S_4 and S_6 share the same six tables and co-appear in 23 of 29 transactional scopes because they implement two sides of the same delegation chain: `TradeSLSBBean` (in S_6) calls `TradeDirect` (in S_4) inside the same transaction for every trading operation. In a microservice architecture, this split would require either: (a) distributed transactions to preserve atomicity across S_4 and S_6 , or (b) abandoning transactional guarantees for 23 of 29 user-visible trading operations. The metric surfaces this obligation precisely because both J_{acc} and J_{tx} are simultaneously high.

References

1. Evans, E.: Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley (2004)
2. Fleischmann, A., Schmidt, W., Stary, C., Obermeier, S., Börger, E.: Subject-Oriented Business Process Management. Springer Berlin Heidelberg (2012), <https://books.google.com.ar/books?id=DdtCDwAAQBAJ>
3. Gerber, M., Chai, J.Y.: Semantic role labeling of implicit arguments for nominal predicates. *Computational Linguistics* **38**(4), 755–798 (12 2012). https://doi.org/10.1162/COLI_a_00110, https://doi.org/10.1162/COLI_a00110
4. Meyers, A., Reeves, R., Macleod, C., Szekely, R., Zielinska, V., Young, B., Grishman, R.: The NomBank project: An interim report. In: Proceedings of the Workshop Frontiers in Corpus Annotation at HLT-NAACL 2004. pp. 24–31. Association for Computational Linguistics, Boston, Massachusetts, USA (May 2 - May 7 2004), <https://aclanthology.org/W04-2705/>
5. Shi, P., Lin, J.J.: Simple bert models for relation extraction and semantic role labeling. *ArXiv abs/1904.05255* (2019)
6. Vera-Rivera, F.H., Puerto, E., Astudillo, H., Gaona, C.M.: Microservices backlog—a genetic programming technique for identification and evaluation of microservices from user stories. *IEEE Access* **9**, 117178–117203 (2021). <https://doi.org/10.1109/ACCESS.2021.3106342>